3DFX Shoelace Formula

How does the Glide driver library calculate the triangle parameters that it sends to the Voodoo 1 and 2 PC graphics accelerator boards?

By Haggarman, 4/28/2026

From the Voodoo and Voodoo 2 datasheets, we can see that the 6 values of the X and Y coordinates of Vertices A, B, and C are all sent directly. The clarification I must make here is that they are sorted by Glide before sending, so that A is on top, B is in the middle, and C is on the bottom.

There are also a set of mirror registers that can accept single precision (32-bit) floating point numbers directly. The float value written automatically triggers an internal conversion to fixed-point format. Since PCI bus writes are relatively slow, this is not a performance burden.

The remaining numeric registers in this parameter block have a consistent description:

- Starting Vertex A ____ information
- Change in ____ with respect to X
- Change in ____ with respect to Y

This conflicts with this statement on Page 17 of the Voodoo 2 datasheet:

> The host provides the starting Gouraud shading RGBA as well as the starting texture S/W, T/W, 1/W , and Voodoo2 Graphics automatically calculates their slopes ΔRGBA, Δ(S/W), Δ(T/W), and Δ(1/W) required for triangle iteration.

I'm sure somewhere there is a handwaving disclaimer about the Voodoo graphics system being a combination of the Glide library that runs on the system CPU and the hardware on the plug-in Voodoo Graphics board. But it is the evident that these RGBA and STW slopes must be pre-calculated before the triangle drawing starts.

The natural way to express the triangle attributes like color are at the 3 vertices, but that's not what the hardware receives. It only receives the Vertex A value directly. Vertex attributes B and C are indirect, as part of the "with respect to X" and "with respect to Y". For clarity, color calculation at an arbitrary (column, row) must look something like this internally:

- Red = **startR** + **dRdY** * (row − vertexA.y) + **dRdX** * (column − vertexA.x)
- Green = **startG** + **dGdY** * (row − vertexA.y) + **dGdX** * (column − vertexA.x)
- Etc.

Start is self-explanatory, but these **d?dX**and **d?dY** parameters remain a mystery because the datasheet does not explain how to calculate them.

There is another curious **triangleCMD** register. It serves to trigger the triangle drawing when written to. It receives a value termed "**AREA**" that is defined very specifically with mathematical equations, but then a note clarifies that only the sign bit is used. My initial thought was, this seems a rather slow way to calculate a single bit that says which way the triangle is facing. But, as it turns out, this strange area calculation is a huge piece of the puzzle. The calculation is as follows:

```
1. dxAB = A.x – B.x
2. dyBC = B.y – C.y
3. dxBC = B.x – C.x
4. dyAB = A.y – B.y
5. AREA = ((dxAB * dyBC) – (dxBC * dyAB)) / 2
```

Combining the above, it looks like this:

```
6. AREA = 0.5 * (((A.x – B.x) * (B.y – C.y)) – ((B.x – C.x) * (A.y – B.y)))
```

The formula above calculates the signed area of a triangle defined by three vertices A, B, and C in a 2-dimensional coordinate system. This is an uncommon form of the shoelace formula. It can also be thought of a cross product of two vectors.

The result of the inner calculation (ignoring taking half) can be positive or negative depending on the radial order of the points (clockwise vs. counter-clockwise).

- If the value is positive, the vertices are ordered counter-clockwise.
- If the value is negative, the vertices are ordered clockwise.
- If the result is zero, the three points are collinear (they lie on a single straight line).

Why the 4 deltas are broken out that way? The more common form of the shoelace area formula is as follows:

```
7. AREA = 0.5 * | A.x * (B.y – C.y) + B.x * (C.y – A.y) + C.x * (A.y – B.y) |
```

Written here for the sake of discussion, but note that this form is NOT USED. One answer could be that it has one more multiplication, but that's still not actually it.

So, I've been stuck on this for years. The tease of these "4 gradients" was there, but I couldn't put it together. I've drawn out various A-B-C triangles on notebook paper. So, I searched for, and found the source code for GLIDE.

Like many things after 30 or so years, the repository has been moved, torn apart, modified, files and folders duplicated, re-bundled, and repackaged. The codebase itself riddled with #ifdefs and hardware bug workarounds and makefiles with flags that were supposed to make compilation easier, but end up polluting the whole thing. As I feared but hoped wouldn't be true, the critical part is written in x86 assembly language.

glide-devel-sezero glide / glide2x / sst1 / glide / src / xdraw96.asm

Code Blame

```
Area_Computation:
; 47-3
; jmp ret_pop0f
    flds    [fa + X]            ;  xa
    fsubs   [fb + X]            ;  dxAB
    flds    [fb + X]            ;  |    xb
    fsubs   [fc + X]            ;  |    dxBC
    flds    [fb + Y]            ;  |    |    yb
    fsubs   [fc + Y]            ;  |    |    dyBC
    flds    [fa + Y]            ;  |    |    |    ya
    fsubs   [fb + Y]            ;  |    |    |    dyAB
    fld     st3                 ;  |    |    |    |    dxAB
    fmul    st0, st2            ;  |    |    |    |    t0         t0=dxAB*dyBC
    fld     st3                 ;  |    |    |    |    |    dxBC
    fmul    st0, st2            ;  |    |    |    |    |    t1    t1=dxBC*dyAB
    fsubp   st1,st0             ;  |    |    |    |    area

    fst     dword [zArea]       ;  |    |    |    |    area

    ; Zero Area Triangle Check
    mov     tmpy, [zArea]      ; j = *(long *)&area
    and     tmpy, 7fffffffh    ; if ((j & 0x7FFFFFFF) == 0)
    jz      zero_area
```

Luckily there is enough similarity that this now familiar AREA calculation jumped out at me the first time this scrolled by. Thank you so much for the code comments!

The code continues, as I transcribe:

```
OOA = 1 / AREA
gradient_dxAB = dxAB * OOA
gradient_dyBC = dyBC * OOA
gradient_dxBC = dxBC * OOA
gradient_dyAB = dyAB * OOA
For each parameter in {R, G, B, A, S, T W} do:
  dpAB = parameter.A – parameter.B
  dpBC = parameter.B – parameter.C
  parameter_start_value = parameter.A
  delta_parameter_dx = dpAB * gradient_dyBC – dpBC * gradient_dyAB
  delta_parameter_dy = dpBC * gradient_dxAB – dpAB * gradient_dxBC
loop
triangleCMD = AREA
```

At line 1 above, OOA is “one over area”, the reciprocal to avoid a much slower division.

And here again are the 4 deltas from the datasheet, this time each effectively divided by the area.

Then another round at a 2x2 matrix multiplication for each of the attributes related to vertices B and C.

And finally, lines 10 and 11 solve the mystery of the “Change in value with respect to X and Y” parameters. Mystery solved for calculating the **d?dX** and **d?dY** parameters listed in the datasheet.

While I wish I could have reasoned this out with geometry, it I didn’t happen that way. However, I am grateful that this Glide source code was open sourced in 1999 so that everyone can benefit.

There are other questions to be solved, like how well are negative numbers handled, likely with wraparound and rollover hazards? How does the edge stepper state machine inside the frame buffer chip even use the (x, y) points without directly using long division in the calculation? How precise are the undocumented internal fixed-point accumulators? But alas, somebody else can take on the tasks to (re)discover these and more.

At the time of this writing, the Glide source code is available at:

https://github.com/sezero/glide

The relevant source code files to this paper are the following subfolder:

```
glide/glide2x/sst1/glide/src/
```

The assembly file is named: `xdraw.asm`

The original programmers graciously provided a C equivalent file named: `gxdraw.c`

Within that C file is this very wise observation about assembly:

```
**  This routine does all the setup needed for drawing a triangle.  It
**  is intended to be an exact specification for the mechanisim used
**  to pass vertices to the assembly language triangle setup code, and
**  as such has no optimizations at all.  Whenever a 'shortcut'
**  routine to draw triangles (such as for antialiasing, fast lines,
**  fast spans, polygons, etc) is needed, this code should be used as
**  the starting point.
```

The answers are out there; you must keep searching for them.